

QuickDelivery — Documentação

Aluno: Joaquim Vilela **Sprint:** 1 – Arquitetura e Backend REST

1. Proposta do Domínio

1.1 Descrição

O **QuickDelivery** é uma plataforma de entregas sob demanda que conecta dois perfis distintos de usuário: o **cliente**, que solicita o transporte de um item de um endereço de origem para um endereço de destino, e o **prestador de serviços** (entregador), que aceita a demanda e executa a entrega. O sistema implementa o ciclo de vida completo de uma solicitação, criação, aceite por um entregador, execução e conclusão (ou cancelamento), com propagação assíncrona das mudanças de estado às partes envolvidas.

1.2 Justificativa

- **Distinção clara entre cliente e prestador**, exigida no enunciado (Seção 2.3). Cliente e entregador têm papéis, telas e fluxos distintos, justificando dois aplicativos móveis independentes.
- **Comunicação naturalmente assíncrona**, alinhada ao princípio central de EDA. Eventos como "nova entrega criada", "entrega aceita" e "status atualizado" são eventos reais de negócio, o que torna o uso de MOM uma escolha justificada.
- **Domínio amplamente compreendido**, baseado em plataformas de delivery, permitindo foco nos aspectos arquiteturais (REST, EDA, MOM, Clean Architecture) sem que a complexidade do negócio se torne barreira.
- **Escopo controlado**, com fluxo principal curto e bem definido (criar → aceitar → em andamento → entregue), viável dentro do prazo do projeto.

1.3 Perfis de Usuário

Cliente (usuário final). Solicita uma entrega informando endereço de coleta, destino e descrição do item; acompanha o status da solicitação e é notificado quando o entregador a aceita.

Prestador de Serviços (entregador). Recebe novas demandas de entrega de forma assíncrona, aceita ou recusa solicitações pendentes e atualiza o status ao longo da execução (em andamento → entregue).

1.4 Principais Funcionalidades

Cliente

- Solicitar uma nova entrega informando origem, destino e descrição do item.
- Listar suas entregas e visualizar o status atual de cada uma.
- Receber notificação quando o status de uma entrega muda.
- Cancelar uma entrega que ainda não foi concluída.

Prestador de Serviços

- Listar entregas pendentes e em andamento.
- Receber notificação assíncrona sempre que uma nova entrega é criada.
- Aceitar ou recusar entregas pendentes.
- Atualizar o status da entrega ao longo da execução.
- Cancelar uma entrega já aceita, quando necessário.

2. Backend REST

O backend expõe 9 endpoints REST (mínimo exigido pelo enunciado: 4) que cobrem o CRUD de clientes, prestadores e entregas, além do endpoint específico `PATCH /deliveries/:id/status` para transição de estado, que valida a máquina de estados do domínio. Quando o status alvo é `ACCEPTED`, o `providerId` é obrigatório, vinculando a entrega ao entregador que a aceitou. As respostas seguem o padrão HTTP: `201` para criação, `200` para leitura/atualização, `400` para erros de validação (campo ausente, transição inválida) e `404` para entidades inexistentes. Erros uniformes no formato `{ "error": "mensagem" }` via middleware central.

3. Organização do Código (Clean Architecture)

O backend é organizado em camadas que isolam responsabilidades. As **rotas** apenas declaram o mapeamento HTTP (verbo + path → controller). Os **controllers** extraem dados de `req`, delegam ao serviço correspondente e devolvem a resposta, não conhecem Prisma nem regras de negócio. Os **services** concentram a lógica de domínio (validações de payload, verificação de existência de entidades relacionadas e validação das transições de status) e desconhecem HTTP. Os **repositories** são a única camada que conhece o Prisma, encapsulando as chamadas ao banco; isso permite trocar de ORM no futuro com impacto localizado. A **infraestrutura** (singleton do `PrismaClient`) e os **middlewares** (tratamento centralizado de erros) ficam à parte, sustentando o restante. Essa separação segue os princípios de Clean Architecture discutidos na ementa e mantém cada camada enxuta, sem abstrações criadas para hipóteses futuras.

4. Validação via Postman

A coleção `postman/QuickDelivery.postman_collection.json` cobre o fluxo completo do domínio: criar cliente → criar prestador → criar entrega → aceitar → em andamento → entregue. Para validar, basta importar a coleção no Postman ou Insomnia e executar as requisições em ordem (ou usar o *Runner* para rodar a coleção inteira). A coleção utiliza variáveis (`baseUrl`, `clientId`, `providerId`, `deliveryId`) com scripts de teste que persistem os IDs entre requisições, dispensando edição manual entre passos.